

Programmazione in Java

Tutor-4: Streams e File I/O

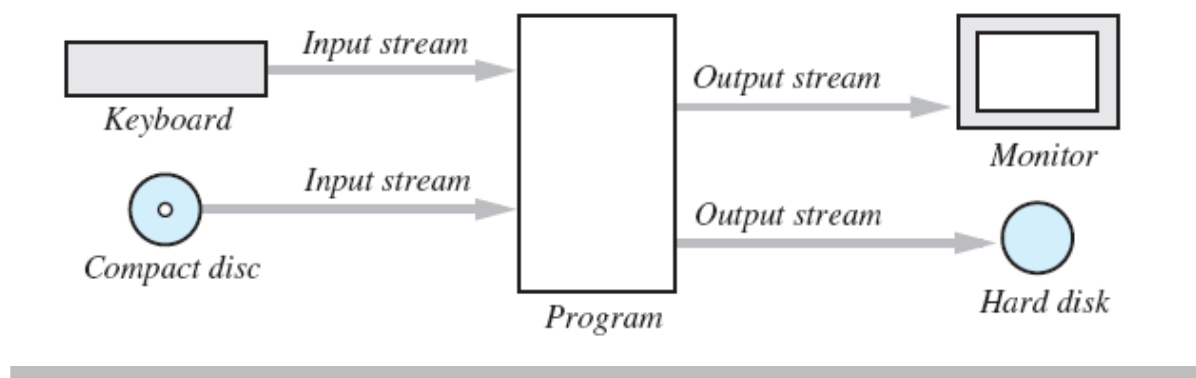
The Concept of a Stream

- Use of files
 - Store Java classes, programs
 - Store pictures, music, videos
 - Can also use files to store program I/O
- A *stream* is a flow of input or output data
 - Characters
 - Numbers
 - Bytes

The Concept of a Stream

- Streams are implemented as objects of special stream classes
 - Class **Scanner**
 - Object **System.out**

I/O Streams



Why Use Files for I/O

- Keyboard input, screen output deal with temporary data
 - When program ends, data is gone
- Data in a file remains after program ends
 - Can be used next time program runs
 - Can be used by another program

Text Files and Binary Files

- All data in files stored as binary digits
 - Long series of zeros and ones
- Files treated as sequence of characters called *text files*
 - Java program source code
 - Can be viewed, edited with text editor
- All other files are called *binary files*
 - Movie, music files
 - Access requires specialized program

Text Files and Binary Files

- Figure 14.2 A text file and a binary file containing the same values

A text file

1	2	3	4	5		-	4	0	2	7		8		...
---	---	---	---	---	--	---	---	---	---	---	--	---	--	-----

A binary file

12345	-4072	8	...
-------	-------	---	-----

Creating a Text File

- Class **PrintWriter** defines methods needed to create and write to a text file
 - Must import package **java.io**
- To open the file
 - Declare *stream variable* for referencing the stream
 - Invoke **PrintWriter** constructor, pass file name as argument
 - Requires **try** and **catch** blocks

Creating a Text File

- File is empty initially
 - May now be written to with method `println`
- Data goes initially to memory buffer
 - When buffer full, goes to file
- Closing file empties buffer, disconnects from stream

Creating a Text File

- View [sample program](#)

class EsempioScritturaFileDiTesto

```

public class EsempioScritturaFileDiTesto {
    public static void main(String[] args) {
        String nomeFile = "out.txt"; //Il nome potrebbe anche
                                     //essere letto da tastiera

        PrintWriter outputStream = null;
        try {
            outputStream = new PrintWriter(nomeFile);
        } catch (FileNotFoundException e) {
            System.out.println("Errore nell'apertura del file " + nomeFile);
            System.exit(0);
        }

        System.out.println("Inserire tre righe di testo:");
        Scanner tastiera = new Scanner(System.in);
        for (int contatore = 1; contatore <= 3; contatore++) {
            String riga = tastiera.nextLine();
            outputStream.println(contatore + " " + riga);
        }
        outputStream.close();
        System.out.println("Le righe sono state scritte su " + nomeFile);
    }
}

```

Creating a Text File

- When creating a file
 - Inform the user of ongoing I/O events, program should not be "silent"
- A file has two names in the program
 - File name used by the operating system
 - The stream name variable
- Opening, writing to file overwrites pre-existing file in directory

Appending to a Text File

- Opening a file new begins with an empty file
 - If already exists, will be overwritten
- Some situations require appending data to existing file
- Command could be

```
OutputStream =  
    new PrintWriter(  
        new FileOutputStream(fileName, true));
```
- Method `println` would append data at end

Reading from a Text File

- Note [text file reading program](#)
class FileDiTestoInputConScannerDemo
- Reads text from file, displays on screen
- Note
 - Statement which opens the file
 - Use of **Scanner** object
 - Boolean statement which reads the file and terminates reading loop

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FileDiTestoInputConScannerDemo {
    public static void main(String[] args) {
        String nomeFile = "out.txt";
        Scanner inputStream = null;
        System.out.println("Il file " + nomeFile + "\ncontiene le righe seguenti:\n");

        try {
            inputStream = new Scanner(new File(nomeFile));
        } catch (FileNotFoundException e) {
            System.out.println("Errore nell'apertura del file " + nomeFile);
            System.exit(0);
        }
        while (inputStream.hasNextLine()) {
            String riga = inputStream.nextLine();
            System.out.println(riga);
        }
        inputStream.close();
    }
}
```

Reading from a Text File

- Additional methods in class **Scanner**

Scanner_Object_Name.hasNext()

Returns true if more input data is available to be read by the method next.

Scanner_Object_Name.hasNextDouble()

Returns true if more input data is available to be read by the method nextDouble.

Scanner_Object_Name.hasNextInt()

Returns true if more input data is available to be read by the method nextInt.

Scanner_Object_Name.hasNextLine()

Returns true if more input data is available to be read by the method nextLine.

Techniques for Any File

- The Class **File**
- Programming Example: Reading a File Name from the Keyboard
- Using Path Names
- Methods of the Class **File**
- Defining a Method to Open a Stream

The Class **File**

- Class provides a way to represent file names in a general way
 - A **File** object represents the name of a file
- The object
new File ("treasure.txt")
is not simply a string
 - It is an object that *knows* it is supposed to name a file

```
import java.util.Scanner;
import java.io.File;
import java.io.FileNotFoundException;

public class FileDiTestoInputConScannerDemo2 {
    public static void main(String[] args) {
        System.out.print("Inserire il nome di un file: ");
        Scanner tastiera = new Scanner(System.in);
        String nomeFile = tastiera.next();
        Scanner inputStream = null;
        System.out.println("Il file " + nomeFile + "\n" +
                           "contiene le righe seguenti:\n");

        try {
            inputStream = new Scanner(new File(nomeFile));
        } catch (FileNotFoundException e) {
            System.out.println("Errore nell'apertura del file " + nomeFile);
            System.exit(0);
        }
        while (inputStream.hasNextLine()) {
            String riga = inputStream.nextLine();
            System.out.println(riga);
        }
        inputStream.close();
    }
}
```

Using Path Names

- Files opened in our examples assumed to be in same folder as where program run
- Possible to specify path names
 - Full path name
 - Relative path name
- Be aware of differences of pathname styles in different operating systems

Methods of the Class File

- Recall that a **File** object is a system-independent abstraction of file's path name
- Class **File** has methods to access information about a path and the files in it
 - Whether the file exists
 - Whether it is specified as readable or not
 - Etc.

Methods of the Class File

- Some methods in class **File**

`public boolean canRead()`

Tests whether the program can read from the file.

`public boolean canWrite()`

Tests whether the program can write to the file.

`public boolean delete()`

Tries to delete the file. Returns true if it was able to delete the file.

`public boolean exists()`

Tests whether an existing file has the name used as an argument to the constructor when the File object was created.

`public String getName()`

Returns the name of the file. (Note that this name is not a path name, just a simple file name.)

`public String getPath()`

Returns the path name of the file.

`public long length()`

Returns the length of the file, in bytes.

Defining a Method to Open a Stream

- Method will have a **String** parameter
 - The file name
- Method will return the stream object
- Will throw exceptions
 - If file not found
 - If some other I/O problem arises
- Should be invoked inside a **try** block and have appropriate **catch** block

Defining a Method to Open a Stream

- Example code

```
public static PrintWriter openOutputTextFile(String fileName)
    throws FileNotFoundException, IOException
{
    PrintWriter toFile = new PrintWriter(fileName);
    return toFile;
}
```

- Example call

```
PrintWriter outputStream = null;
try
{
    outputStream = openOutputTextFile("data.txt");
}
< appropriate catch block(s) >
```

Creating a Binary File

- Stream class **ObjectOutputStream** allows files which can store
 - Values of primitive types
 - Strings
 - Other objects
- View [program which writes integers](#),
class FileBinarioOutputDemo

```
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.Scanner;

public class FileBinarioOutputDemo {
    public static void main(String[] args) {
        String nomeFile = "numeri.dat";
        try {
            ObjectOutputStream outputStream = new ObjectOutputStream(new FileOutputStream(nomeFile));
            Scanner tastiera = new Scanner(System.in);
            System.out.println("Inserire degli interi non negativi.");
            System.out.println("Inserire un numero negativo per terminare.");

            int unIntero;
            do {
                unIntero = tastiera.nextInt();
                outputStream.writeInt(unIntero);
            } while (unIntero >= 0);

            System.out.println("I numeri e il valore di terminazione");
            System.out.println("sono stati scritti nel file " + nomeFile);
            outputStream.close();
        } catch (FileNotFoundException e) {
            System.out.println("Errore nell'apertura del file " + nomeFile);
        } catch (IOException e) {
            System.out.println("Errore nella scrittura nel file " + nomeFile);
        }
    }
}
```

Writing Primitive Values to a Binary File

- Method `println` not available
 - Instead use `writeln` method
- Binary file stores numbers in binary form
 - A sequence of bytes
 - One immediately after another

This file is a binary file. You cannot read this file using a text editor.

1	2	3	-1
---	---	---	----

*The **-1** in this file is a sentinel value. Ending a file with a sentinel value is not essential, as you will see later.*

Creating a Binary File

- Note the line to open the file
 - Constructor for **ObjectOutputStream** cannot take a **String** parameter
 - Constructor for **FileOutputStream** can

Writing Primitive Values to a Binary File

- Some methods in class **ObjectOutputStream**

```
public ObjectOutputStream(OutputStream streamObject)
```

Creates an output stream that is connected to the specified binary file. There is no constructor that takes a file name as an argument. If you want to create a stream by using a file name, you write either

```
new ObjectOutputStream(new FileOutputStream(File_Name))
```

or, using an object of the class `File`,

```
new ObjectOutputStream(new FileOutputStream(  
    new File(File_Name)))
```

Either statement creates a blank file. If there already is a file named *File_Name*, the old contents of the file are lost.

The constructor for `FileOutputStream` can throw a `FileNotFoundException`. If it does not, the constructor for `ObjectOutputStream` can throw an `IOException`.

```
public void writeInt(int n) throws IOException
```

Writes the `int` value `n` to the output stream.

```
public void writeLong(long n) throws IOException
```

Writes the `long` value `n` to the output stream.

Writing Primitive Values to a Binary File

- Some methods in class **ObjectOutputStream**

<pre>public void writeDouble(double x) throws IOException</pre> Writes the <code>double</code> value <code>x</code> to the output stream.
<pre>public void writeFloat(float x) throws IOException</pre> Writes the <code>float</code> value <code>x</code> to the output stream.
<pre>public void writeChar(int c) throws IOException</pre> Writes a <code>char</code> value to the output stream. Note that the parameter type of <code>c</code> is <code>int</code>. However, Java will automatically convert a <code>char</code> value to an <code>int</code> value for you. So the following is an acceptable invocation of <code>writeChar</code>: <pre>outputStream.writeChar('A');</pre>
<pre>public void writeBoolean(boolean b) throws IOException</pre> Writes the <code>boolean</code> value <code>b</code> to the output stream.
<pre>public void writeUTF(String aString) throws IOException</pre> Writes the string <code>aString</code> to the output stream. UTF refers to a particular method of encoding the string. To read the string back from the file, you should use the method <code>readUTF</code> of the class <code>ObjectInputStream</code>. These topics are discussed in the next section.

Writing Primitive Values to a Binary File

- Some methods in class **ObjectOutputStream**

```
public void writeObject(Object anObject) throws IOException,  
                                           NotSerializableException, InvalidClassException  
Writes anObject to the output stream. The argument should be an object of a serial-  
izable class, a concept discussed later in this chapter. Throws a NotSerializable-  
Exception if the class of anObject is not serializable. Throws an  
InvalidClassException if there is something wrong with the serialization. The  
method writeObject is covered later in this chapter.
```

```
public void close() throws IOException  
Closes the stream's connection to a file.
```

Writing Strings to a Binary File

- Use method `writeUTF`

- Example

```
outputStream.writeUTF("Hi Mom");
```

- UTF stands for *Unicode Text Format*
- Uses a varying number of bytes to store different strings
 - Depends on length of string
 - Contrast to `writeInt` which uses same for each

Reading from a Binary File

- File must be opened as an **ObjectInputStream**
- Read from binary file using methods which correspond to write methods
 - Integer written with **writeInt** will be read with **readInt**
- Be careful to read same type as was written

```

import java.io.FileInputStream;
import java.io.ObjectInputStream;
import java.io.EOFException;
import java.io.FileNotFoundException;
import java.io.IOException;

public class FileBinarioInputDemo {
    public static void main(String[] args) {
        String nomeFile = "numeri.dat";
        try {
            ObjectInputStream inputStream = new ObjectInputStream(new
                FileInputStream(nomeFile));
            System.out.println("Lettura dei numeri non negativi");
            System.out.println("nel file " + nomeFile);

            int unIntero = inputStream.readInt();
            while (unIntero >= 0) {
                System.out.println(unIntero);
                unIntero = inputStream.readInt();
            }

            System.out.println("Fine della lettura dal file.");
            inputStream.close();
        } catch (FileNotFoundException e) {
            System.out.println("Errore nell'apertura del file " + nomeFile);
        } catch (EOFException e) {
            System.out.println("Errore nella lettura del file " + nomeFile);
            System.out.println("Raggiunta la fine del file.");
        } catch (IOException e) {
            System.out.println("Errore nella lettura del file " + nomeFile);
        }
    }
}

```

Reading from a Binary File

- Some methods of class **ObjectInputStream**

`ObjectInputStream(InputStream streamObject)`

Creates an input stream that is connected to the specified binary file. There is no constructor that takes a file name as an argument. If you want to create a stream by using a file name, you use either

```
new ObjectInputStream(new FileInputStream(File_Name))
```

or, using an object of the class `File`,

```
new ObjectInputStream(new FileInputStream(  
    new File(File_Name)))
```

The constructor for `FileInputStream` can throw a `FileNotFoundException`. If it does not, the constructor for `ObjectInputStream` can throw an `IOException`.

`public int readInt() throws EOFException, IOException`

Reads an `int` value from the input stream and returns that `int` value. If `readInt` tries to read a value from the file that was not written by the method `writeInt` of the class `ObjectOutputStream` (or was not written in some equivalent way), problems will occur. If the read goes beyond the end of the file, an `EOFException` is thrown.

Reading from a Binary File

- Some methods of class **ObjectInputStream**

`public long readLong() throws EOFException, IOException`

Reads a `long` value from the input stream and returns that `long` value. If `readLong` tries to read a value from the file that was not written by the method `writeLong` of the class `ObjectOutputStream` (or was not written in some equivalent way), problems will occur. If the read goes beyond the end of the file, an `EOFException` is thrown.

Note that you cannot write an integer using `writeLong` and later read the same integer using `readInt`, or to write an integer using `writeInt` and later read it using `readLong`. Doing so will cause unpredictable results.

`public double readDouble() throws EOFException, IOException`

Reads a `double` value from the input stream and returns that `double` value. If `readDouble` tries to read a value from the file that was not written by the method `writeDouble` of the class `ObjectOutputStream` (or was not written in some equivalent way), problems will occur. If the read goes beyond the end of the file, an `EOFException` is thrown.

Reading from a Binary File

- Some methods of class **ObjectInputStream**

`public float readFloat() throws EOFException, IOException`

Reads a `float` value from the input stream and returns that `float` value. If `readFloat` tries to read a value from the file that was not written by the method `writeFloat` of the class `ObjectOutputStream` (or was not written in some equivalent way), problems will occur. If the read goes beyond the end of the file, an `EOFException` is thrown.

Note that you cannot write a floating-point number using `writeDouble` and later read the same number using `readFloat`, or write a floating-point number using `writeFloat` and later read it using `readDouble`. Doing so will cause unpredictable results, as will other type mismatches, such as writing with `writeInt` and then reading with `readFloat` or `readDouble`.

Reading from a Binary File

- Some methods of class **ObjectInputStream**

```
public char readChar() throws EOFException, IOException
```

Reads a char value from the input stream and returns that char value. If readChar tries to read a value from the file that was not written by the method writeChar of the class ObjectOutputStream (or was not written in some equivalent way), problems will occur. If the read goes beyond the end of the file, an EOFException is thrown.

```
public boolean readBoolean() throws EOFException, IOException
```

Reads a boolean value from the input stream and returns that boolean value. If readBoolean tries to read a value from the file that was not written by the method writeBoolean of the class ObjectOutputStream (or was not written in some equivalent way), problems will occur. If the read goes beyond the end of the file, an EOFException is thrown.

Reading from a Binary File

- Some methods of class **ObjectInputStream**

```
public String readUTF() throws IOException,  
                           UTFDataFormatException
```

Reads a **String** value from the input stream and returns that **String** value. If **readUTF** tries to read a value from the file that was not written by the method **writeUTF** of the class **ObjectOutputStream** (or was not written in some equivalent way), problems will occur. One of the exceptions **UTFDataFormatException** or **IOException** can be thrown.

```
Object readObject() throws ClassNotFoundException,  
                        InvalidClassException, OptionalDataException, IOException
```

Reads an object from the input stream. Throws a **ClassNotFoundException** if the class of a serialized object cannot be found. Throws an **InvalidClassException** if something is wrong with the serializable class. Throws an **OptionalDataException** if a primitive data item, instead of an object, was found in the stream. Throws an **IOException** if there is some other I/O problem. The method **readObject** is covered in Section 10.5.

```
public void close() throws IOException
```

Closes the stream's connection to a file.

The Class **EOFException**

- Many methods that read from a binary file will throw an **EOFException**
 - Can be used to test for end of file
 - Thus can end a reading loop
- View [example program](#),
class **EOFExceptionDemo**

```
import java.io.FileInputStream;
import java.io.ObjectInputStream;
import java.io.EOFException;
import java.io.FileNotFoundException;
import java.io.IOException;

public class EOFExceptionDemo {
    public static void main(String[] args) {
        String nomeFile = "numeri.dat";
        try {
            ObjectInputStream inputStream = new ObjectInputStream(new
                FileInputStream(nomeFile));
            System.out.println("Lettura di TUTTI gli interi");
            System.out.println("nel file " + nomeFile);

            try {
                while (true) {
                    int unIntero = inputStream.readInt();
                    System.out.println(unIntero);
                }
            } catch (EOFException e) {
                System.out.println("Fine della lettura dal file.");
            }
            inputStream.close();
        } catch (FileNotFoundException e) {
            System.out.println("Errore nell'apertura del file " + nomeFile);
        } catch (IOException e) {
            System.out.println("Errore nella lettura del file " + nomeFile);
        }
    }
}
```

Programming Example

- Processing a file of binary data
 - Asks user for 2 file names
 - Reads numbers in input file
 - Doubles them
 - Writes them to output file
- View [processing program](#), listing 14.9
class **Doubler**

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.EOFException;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.Scanner;

public class Raddoppia {
    private ObjectInputStream inputStream = null;
    private ObjectOutputStream outputStream = null;

    /**
     Raddoppia gli interi in un file e scrive il risultato in un altro file.
     */
    public static void main(String[] args) {
        Raddoppia moltiplicatore = new Raddoppia();
        moltiplicatore.collegaFileDiInput();
        moltiplicatore.collegaFileDiOutput();
        moltiplicatore.moltiplicaPerDue();
        moltiplicatore.chiudiFile();

        System.out.println("Numeri letti da un file di input");
        System.out.println("raddoppiati e copiati in un file di output.");
    }
}
```

```
public void collegaFileDiInput() {  
    String nomeFileDiInput = leggiNomeFile("Inserisci il nome del file di input:");  
    try {  
        inputStream = new ObjectInputStream(new FileInputStream(nomeFileDiInput));  
    } catch (FileNotFoundException e) {  
        System.out.println("File " + nomeFileDiInput + " non trovato.");  
        System.exit(0);  
    } catch (IOException e) {  
        System.out.println("Errore nell'apertura del file di input " +  
                             nomeFileDiInput);  
        System.out.println(e.getMessage());  
        System.exit(0);  
    }  
}
```

```
public void collegaFileDiOutput() {  
    String nomeFileDiOutput = leggiNomeFile("Inserisci il nome del file di output:");  
    try {  
        outputStream = new ObjectOutputStream(new  
            FileOutputStream(nomeFileDiOutput));  
    } catch (IOException e) {  
        System.out.println("Errore nell'apertura del file di output " +  
                             nomeFileDiOutput);  
        System.out.println(e.getMessage());  
        System.exit(0);  
    }  
}
```

```
private String leggiNomeFile(String messaggio) {  
    String nomeFile = null;  
    System.out.println(messaggio);  
    Scanner tastiera = new Scanner(System.in);  
    nomeFile = tastiera.next();  
    return nomeFile;  
}
```

```
public void moltiplicaPerDue() {  
    try {  
        while (true) {  
            int prossimo = inputStream.readInt();  
            outputStream.writeInt(2 * prossimo);  
        } catch (EOFException e) {  
            //Non fa niente, serve solo per terminare il ciclo.  
        } catch (IOException e) {  
            System.out.println("Errore nella lettura o nella scrittura dei file.");  
            System.out.println(e.getMessage());  
            System.exit(0);  
        }  
    }  
}
```

```
public void chiudiFile() {  
    try {  
        inputStream.close();  
        outputStream.close();  
    } catch (IOException e) {  
        System.out.println("Errore nella chiusura dei file.");  
        System.out.println(e.getMessage());  
        System.exit(0);  
    }  
}
```

Binary-File I/O with Class Objects

- Consider the need to write/read objects other than **Strings**
 - Possible to write the individual instance variable values
 - Then reconstruct the object when file is read
- A better way is provided by Java
 - *Object serialization* – represent an object as a sequence of bytes to be written/read
 - Possible for any class implementing **Serializable**

ESERCIZIO 1

- Si scriva un programma che scriva una poesia (a vostra scelta!) in un file di testo. Il nome del file deve essere chiesto all'utente. Ogni verso della poesia va in una riga.

L'INFINITO

ESEMPIO:

Sempre caro mi fu quest'ermo colle,
E questa siepe, che da tanta parte
Dell'ultimo orizzonte il guardo esclude.
Ma sedendo e mirando, interminati
Spazi di là da quella, e sovrumani
Silenzi, e profondissima quiete
Io nel pensier mi fingo; ove per poco
Il cor non si spaura. E come il vento
Odo stormir tra queste piante, io quello
Infinito silenzio a questa voce
Vo comparando: e mi sovvien l'eterno,
E le morte stagioni, e la presente
E viva, e il suon di lei. Così tra questa
Immensità s'annega il pensier mio:
E il naufragar m'è dolce in questo mare.

ESERCIZIO 2

- Si scriva un programma che registri gli acquisti effettuati in un negozio. Per ogni acquisto, il programma dovrà leggere da tastiera il nome del prodotto, il prezzo e la quantità acquistata. Si calcoli il costo totale della merce acquistata (quantità per prezzo) e lo si scriva in un file di testo, mostrando anche sullo schermo il costo totale dei prodotti acquistati fino a questo momento. Una volta che sono stati registrati tutti gli acquisti, si scriva il costo totale sia sullo schermo che nel file. (Dato che si vuole tenere traccia di tutti gli acquisti effettuati, i dati dovranno essere di volta in volta aggiunti alla fine del file.)